```
const TelegramBot = require('node-telegram-bot-api');
const express = require('express');
const cors = require('cors');

const BOT_TOKEN = process.env.BOT_TOKEN;
const app = express();
app.use(cors());
app.use(express.json());

const bot = new TelegramBot(BOT_TOKEN, { polling: true });

// ─── In-memory store (resets on redeploy — fine for free tier) ───
let messages = [];      // raw messages from groups
let analysis = {        // latest AI analysis result
  subjects: [],
  assignments: [],
  exams: [],
  notes: [],
  lastUpdated: null
};

// ─── Keywords for detection ──────────────────────────────────────
const ASSIGNMENT_WORDS = ['assignment', 'homework', 'submit', 'submission', 'due'
const EXAM_WORDS = ['exam', 'test', 'quiz', 'midterm', 'final', 'viva', 'practica
const NOTE_WORDS = ['lecture', 'notes', 'slide', 'pdf', 'chapter', 'topic', 'syll
const DATE_PATTERN = /(\d{1,2}[\/\-\.]\d{1,2}[\/\-\.]\d{2,4}|\d{1,2}\s*(jan|feb|m
const SUBJECT_PATTERN = /(maths?|math|physics|chemistry|biology|english|history|g

function extractSubject(text) {
  const found = text.match(SUBJECT_PATTERN);
  return found ? [...new Set(found.map(s => capitalize(s.trim())))] : [];
}

function extractDate(text) {
  const found = text.match(DATE_PATTERN);
  return found ? found[0] : null;
}

function capitalize(s) {
  return s.charAt(0).toUpperCase() + s.slice(1).toLowerCase();
}

function classifyMessage(text) {
  const lower = text.toLowerCase();
```

```
  if (ASSIGNMENT_WORDS.some(w => lower.includes(w))) return 'assignment';
  if (EXAM_WORDS.some(w => lower.includes(w))) return 'exam';
  if (NOTE_WORDS.some(w => lower.includes(w))) return 'note';
  return null;
}

function runAnalysis() {
  const assignmentsMap = {};
  const examsMap = {};
  const notesArr = [];
  const subjectsSet = new Set();

  for (const msg of messages) {
    const type = classifyMessage(msg.text);
    const subjects = extractSubject(msg.text);
    const date = extractDate(msg.text);
    subjects.forEach(s => subjectsSet.add(s));

    if (type === 'assignment') {
      const key = (subjects[0] || 'General') + '_' + (date || msg.text.slice(0, 3
      if (!assignmentsMap[key]) {
        assignmentsMap[key] = {
          subject: subjects[0] || 'General',
          text: msg.text.slice(0, 120),
          deadline: date || 'No date mentioned',
          group: msg.groupName,
          time: msg.time,
          done: false
        };
      }
    } else if (type === 'exam') {
      const key = (subjects[0] || 'General') + '_exam_' + (date || '');
      if (!examsMap[key]) {
        examsMap[key] = {
          subject: subjects[0] || 'General',
          text: msg.text.slice(0, 120),
          date: date || 'Date TBC',
          group: msg.groupName,
          time: msg.time
        };
      }
    } else if (type === 'note') {
      notesArr.push({
        subject: subjects[0] || 'General',
        text: msg.text.slice(0, 100),
        group: msg.groupName,
        time: msg.time
```

```
      });
    }
  }

  analysis = {
    subjects: [...subjectsSet],
    assignments: Object.values(assignmentsMap).slice(0, 20),
    exams: Object.values(examsMap).slice(0, 10),
    notes: notesArr.slice(0, 15),
    totalMessages: messages.length,
    lastUpdated: new Date().toISOString()
  };

  console.log(`Analysis updated: ${analysis.assignments.length} assignments, ${an
}

// ─── Listen to ALL group messages ──────────────────────────────
bot.on('message', (msg) => {
  if (!msg.text) return;
  const chatType = msg.chat.type;
  if (chatType !== 'group' && chatType !== 'supergroup') return;

  const entry = {
    id: msg.message_id,
    text: msg.text,
    groupName: msg.chat.title || 'Unknown Group',
    groupId: msg.chat.id,
    sender: msg.from?.first_name || 'Unknown',
    time: new Date(msg.date * 1000).toISOString()
  };

  messages.push(entry);
  // Keep last 500 messages only
  if (messages.length > 500) messages = messages.slice(-500);

  // Re-analyse every 5 new messages
  if (messages.length % 5 === 0) runAnalysis();
});

// ─── Bot commands ──────────────────────────────────────────────
bot.onText(/\/start/, (msg) => {
  bot.sendMessage(msg.chat.id,
    `👋 Hi! I'm Maher's College Bot.\n\nAdd me to your college Telegram groups an
  );
});

bot.onText(/\/status/, (msg) => {
```

```
  bot.sendMessage(msg.chat.id,
    `📊 Maher Bot Status:\n\n💬 Messages tracked: ${messages.length}\n📋 Assignme
  );
});

bot.onText(/\/summary/, (msg) => {
  let text = `📚 College Summary:\n\n`;
  if (analysis.assignments.length) {
    text += `📋 ASSIGNMENTS (${analysis.assignments.length}):\n`;
    analysis.assignments.slice(0, 3).forEach(a => {
      text += `• ${a.subject} — Due: ${a.deadline}\n`;
    });
    text += '\n';
  }
  if (analysis.exams.length) {
    text += `📅 EXAMS (${analysis.exams.length}):\n`;
    analysis.exams.slice(0, 3).forEach(e => {
      text += `• ${e.subject} — ${e.date}\n`;
    });
  }
  if (!analysis.assignments.length && !analysis.exams.length) {
    text += 'No assignments or exams detected yet. Make sure I\'m added to your c
  }
  bot.sendMessage(msg.chat.id, text);
});

// ── API endpoints for Maher app ──────────────────────────────
app.get('/', (req, res) => res.json({ status: 'Maher Bot running ✅', messages: m

app.get('/api/analysis', (req, res) => {
  if (messages.length > 0 && !analysis.lastUpdated) runAnalysis();
  res.json(analysis);
});

app.get('/api/messages', (req, res) => {
  res.json({ total: messages.length, recent: messages.slice(-20) });
});

app.post('/api/mark-done', (req, res) => {
  const { index } = req.body;
  if (analysis.assignments[index]) {
    analysis.assignments[index].done = true;
  }
  res.json({ success: true });
});

app.get('/api/refresh', (req, res) => {
```

```
  runAnalysis();
  res.json({ success: true, analysis });
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, '0.0.0.0', () => console.log(`Maher Bot server running on port $
```